

Some Supervised Classification Algorithms

Gian Marco Miccio

ReLU — NTNU

07/10/2025

Today we will take a look at some algorithms and methods for classification, widely used in machine learning

The lesson will be divided in two, we'll now take a look at 2 models.

- **Logistic Regression**
- **Naive Bayes**

Later on we will move to **Decision Trees** (very important!) and some notes on how to tell if an algorithm is fast or slow (**Computational Complexity**)

We're going to start with some basics today as well, as a refresher

Bernoulli Distribution

$Y \sim \text{Be}(p)$ where $Y \in \{0, 1\}$,

$$\mathbb{P}(Y = i) = \begin{cases} p, & \text{if } i = 1, \\ 1 - p, & \text{if } i = 0. \end{cases}$$

with mean $\mathbb{E}[Y] = p$ and variance $\text{VAR}[Y] = p(1 - p)$

Idea: you can think of this Random Variable as being an unfair **coin flip** (unfair because the probabilities of heads and tails are not the same)

Idea: What if we want to model n consecutive coin tosses? Every coin toss is an independent Bernoulli distributed Random Variable

Binomial Distribution

$Y \sim \text{Bin}(n, p)$ where $Y \in \{0, 1, \dots, n\}$,

$$\mathbb{P}(Y = i) = \binom{n}{i} p^i (1 - p)^{n-i}, \quad i = 0, 1, 2, \dots, n.$$

with mean $\mathbb{E}[Y] = p$ and variance $\text{VAR}[Y] = p(1 - p)$

Idea: A binomial is n coin tosses in a row, and the result of the RV is the number of heads

$$\mathbb{P}(Y = i) = \binom{n}{i} p^i (1 - p)^{n-i}, \quad i = 0, 1, 2, \dots, n.$$

You'll notice this symbol in the formula, what does it actually mean?

$$\binom{n}{i} = \frac{n!}{i! (n - i)!}, \quad i = 0, 1, 2, \dots, n$$

$$\binom{n}{i} = \frac{n!}{i!(n-i)!}, \quad i = 0, 1, 2, \dots, n$$

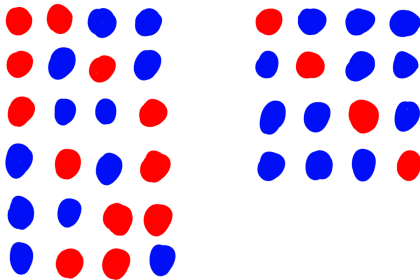
Idea: The results actually come from the Pascal triangle! n is the number of the row from the top, i is the position from the left

			1			
		1		1		
	1		2		1	
	1	3		3		1
	1	4	6	4		1
1	5	10	10	5		1



What these number represent are the total number of possible arrangements of the results you're studying

For example, the total number of arrangements of 2 head and 2 tails occurring vs 3 heads and 1 tail



Note: this is the worst drawing I've ever made

Idea: What if now we think of a coin having more than than two sides?
(Like a die for example). We want to count how many occurrences of each
different side happen

Multinomial Distribution

$\underline{X} \sim \text{Mult}(n, \underline{p})$ where $\underline{X} = (X_1, \dots, X_k)$ and $X_i \in \{0, 1, \dots, n\}$,

$$\mathbb{P}(X_1 = x_1, \dots, X_k = x_k) = \frac{n!}{x_1! x_2! \dots x_k!} p_1^{x_1} p_2^{x_2} \dots p_k^{x_k}, \quad \text{with } \sum_{i=1}^k x_i = n$$

where $\underline{p} = (p_1, \dots, p_k)$ with $\sum_{i=1}^k p_i = 1$.

Idea: The Multinomial is a generalization of the Binomial: instead of counting successes for one event, it counts occurrences across **multiple categories** in n trials.



Example: 6-Sided Die (Multinomial)

Suppose we roll a fair 6-sided die $n = 10$ times. Let $\underline{X} = (X_1, X_2, X_3, X_4, X_5, X_6)$ be the counts of each face appearing. The probabilities for each face are $\underline{p} = (1/6, 1/6, 1/6, 1/6, 1/6, 1/6)$.

If we observe the following counts in 10 rolls:

$$\underline{x} = (2, 1, 3, 0, 2, 2)$$

the probability of this outcome under the multinomial model is

$$P(\underline{X} = \underline{x}) = \frac{10!}{2! 1! 3! 0! 2! 2!} \left(\frac{1}{6}\right)^2 \left(\frac{1}{6}\right)^1 \left(\frac{1}{6}\right)^3 \left(\frac{1}{6}\right)^0 \left(\frac{1}{6}\right)^2 \left(\frac{1}{6}\right)^2$$

Idea: Each X_i counts how many times face i occurs. The factorial term accounts for all possible orderings of the rolls.



we've talked about regression before, when we were predicting values that could be any real number, but...

What do we do when the response is **discrete**?

Example: Breast Cancer dataset

worst perimeter	worst area	worst smoothness	worst compactness	worst concavity	worst concave points	worst symmetry	worst fractal dimension	target
96.05	677.9	0.1426	0.2378	0.2671	0.10150	0.3014	0.08750	1
165.90	1866.0	0.1193	0.2336	0.2687	0.17890	0.2551	0.06589	0
124.90	1156.0	0.1546	0.2394	0.3791	0.15140	0.2837	0.08019	0
89.61	515.8	0.1450	0.2629	0.2403	0.07370	0.2556	0.09359	1
78.78	457.8	0.1345	0.2118	0.1797	0.06918	0.2329	0.08134	1

- the dataset has 30 covariates
- the response "target" can only be 0 or 1



- If the response Y is a **real continuous number**, we are doing **regression**, and we can use models like linear regression, decision trees, gradient boosting
- If the response Y is **discrete**, for example can only have values in $\{0, 1, 2\}$, well then linear regression doesn't really work, we need **different models**

fortunately, a lot of models are flexible and can be adapted to both regression

a little reminder on the terms we are using

- **Supervised Learning:** when we know the true answer for the response in the dataset, meaning we know which class a training data belongs to
- **Unsupervised Learning:** this would be the case in which the classes we are trying to assign are created by us rather than being something that's already been studied about the dataset

What model can we use?

Logistic Regression

Logistic Regression

We've briefly talked about it in past bootcamps, today we're gonna look at how to use it and how it's tied to Linear Regression

A different way of looking at a linear regression is to consider the response Y to have **it's own distribution**

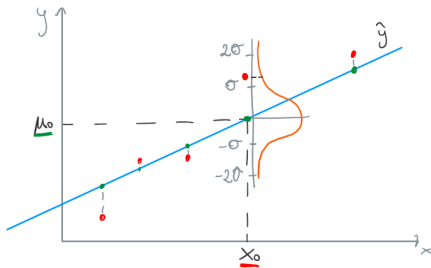
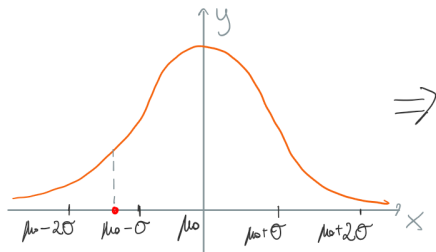
$$Y = \mathbb{X}\underline{\beta} + \underline{\epsilon}$$

where we know $\underline{\epsilon} \sim N(0, \sigma^2)$

$$Y \sim N(\underline{\mu}, \sigma^2 I)$$

Idea: we are grouping the random element directly into Y , basically creating a shorter expression to represent randomness

Logistic Regression



Given this, any Linear Regression model can be thought of having 3 different components

- Random Component
- Systematic Component
- Link Function

The **Random Component** is what we have just discussed, and in element form can be written as

$$Y_i \sim N(\mu_i, \sigma^2)$$

μ_i is our parameter of interest, that ultimately shapes the linear distribution, and it will be different for each observation

A **Systematic Component**, that is, none other than our proposed linear relationship:

$$\eta_i = \mathbf{x}_i^\top \boldsymbol{\beta}$$

It's the linear predictor, where $\mathbf{x}_i$ is a fixed p -dimensional column vector of covariates

Idea: with this component we are introducing the linear relationship and therefore making it a linear regression model



Finally, a **Link function**, that connects the two other components

$$\mu_i = \eta_i$$

Note: this one looks useless, but it is important when dealing with different models.

How is this connected to logistic regression?

We can modify these components to create new models suitable for our problem!

The **Random Component** becomes the distribution of a RV that has only two outcomes, a **Bernoulli**

$$Y_i \sim Be(p_i)$$

where p_i is our parameter of interest

Idea: every observation has it's own chance of being classified as 0 or 1 depending on the values of its covariates. It's as if we were flipping a coin for each observation based on how high we think the chance of it being 1 is

The **Systematic Component** remains the same:

$$\eta_i = \mathbf{x}_i^\top \boldsymbol{\beta}$$

Idea: We still want to use a linear relationship between coefficients and covariates

The **Link function** is where it gets interesting, instead of simply putting our mean equal to the linear relationship, we plug into a different function:

$$\eta_i = \log \left(\frac{p_i}{1 - p_i} \right)$$

Idea: we are still linking the random component and its parameter of interest p_i and the systematic component η_i , just in a different way

$$\eta_i = \log \left(\frac{p_i}{1 - p_i} \right)$$

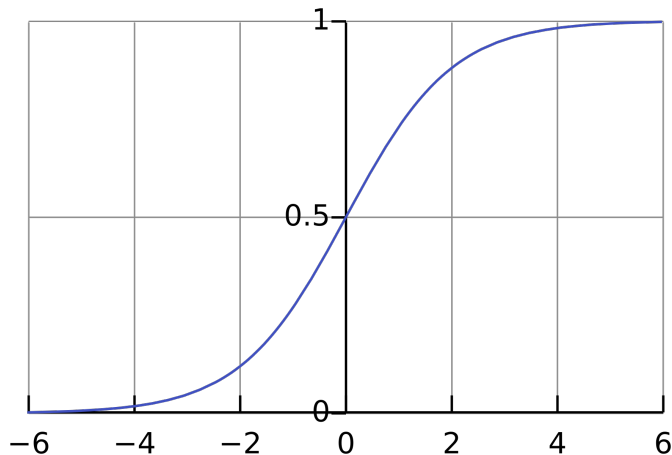
implies

$$p_i = \frac{e^{\eta_i}}{1 + e^{\eta_i}} = \frac{e^{\mathbf{x}_i^\top \boldsymbol{\beta}}}{1 + e^{\mathbf{x}_i^\top \boldsymbol{\beta}}} = \frac{1}{1 + e^{-\mathbf{x}_i^\top \boldsymbol{\beta}}}.$$

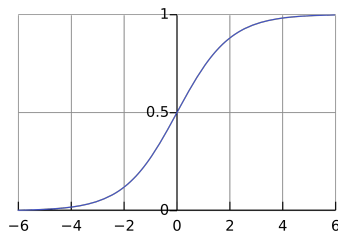
which is a very interesting function we've come across already

Logistic Regression

$$p_i = \frac{1}{1 + e^{-\eta_i}}$$



Logistic Regression

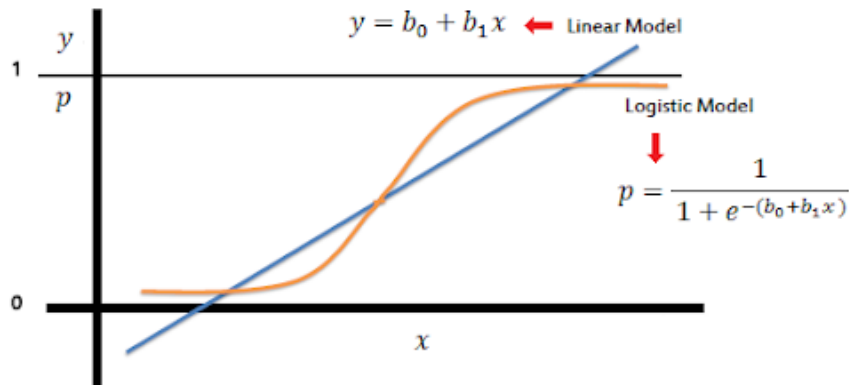


We choose this because it has some nice properties:

- is contained between 0 and 1 (like a probability of an event happening)
- has a nice split at 0.5
- as $x \rightarrow \infty$ the curve has limit 1
- as $x \rightarrow -\infty$ the curve has limit 0



Logistic Regression



Example: Breast Cancer dataset

worst perimeter	worst area	worst smoothness	worst compactness	worst concavity	worst concave points	worst symmetry	worst fractal dimension	target
96.05	677.9	0.1426	0.2378	0.2671	0.10150	0.3014	0.08750	1
165.90	1866.0	0.1193	0.2336	0.2687	0.17890	0.2551	0.06589	0
124.90	1156.0	0.1546	0.2394	0.3791	0.15140	0.2837	0.08019	0
89.61	515.8	0.1450	0.2629	0.2403	0.07370	0.2556	0.09359	1
78.78	457.8	0.1345	0.2118	0.1797	0.06918	0.2329	0.08134	1

Let's fit a logistic regression model on this dataset



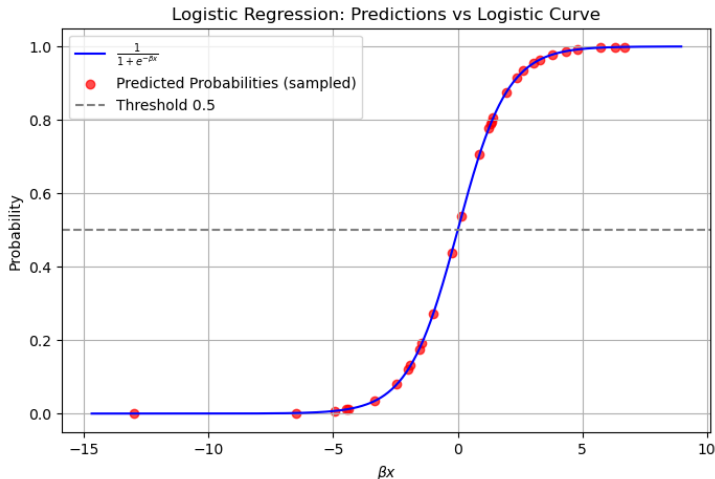
Logistic Regression

How we do it

```
from sklearn.linear_model import LogisticRegression
log_reg = LogisticRegression(max_iter=1000)
log_reg.fit(X_train_scaled, y_train)
y_pred = log_reg.predict(X_test_scaled)
```

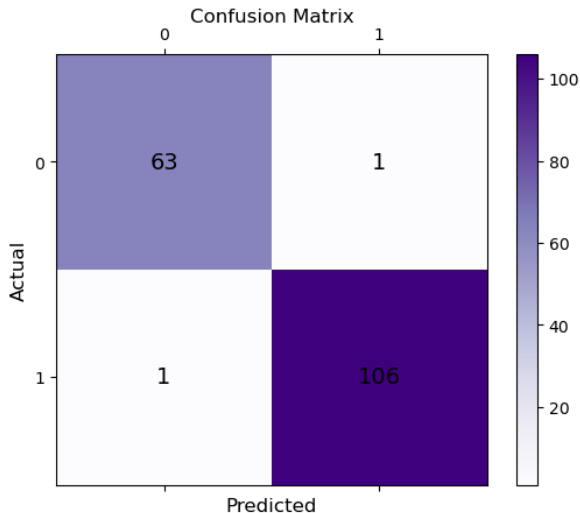
Logistic Regression

Here's the plot of the prediction on the logistic curve



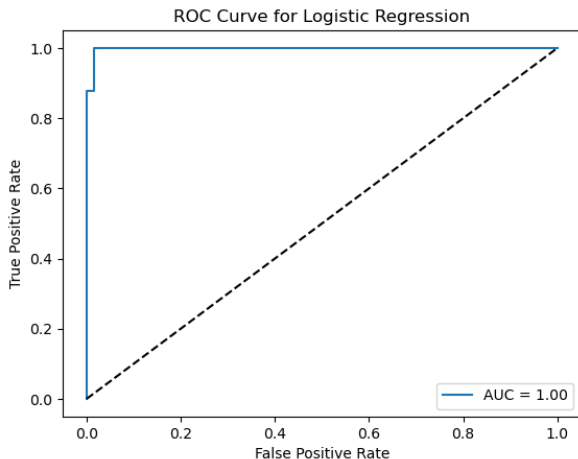
Logistic Regression

Evaluation: Confusion Matrix



Logistic Regression

Evaluation: ROC curve



This is a good model that serves as a base, but relies on the response being **binary**

What do we do when the response is discrete but has **more** than just 2 categories?

sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
6.1	2.8	4.7	1.2	versicolor
5.7	3.8	1.7	0.3	setosa
7.7	2.6	6.9	2.3	virginica
6.0	2.9	4.5	1.5	versicolor
6.8	2.8	4.8	1.4	versicolor
5.4	3.4	1.5	0.4	setosa
5.6	2.9	3.6	1.3	versicolor
6.9	3.1	5.1	2.3	virginica



Naive Bayes

but what is Bayes?

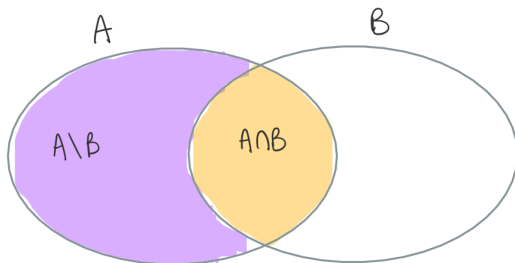
let's take a step back

Probability of A and B

$$\mathbb{P}(A \cap B) = \mathbb{P}(A \text{ and } B)$$

where:

- $\mathbb{P}(A \cap B)$ is the probability that both A and B occur
- we can also deduce $\mathbb{P}(A \setminus B) = \mathbb{P}(A) - \mathbb{P}(A \cap B)$



Example: You're throwing a die and

- A is the event of getting an odd number

$$\mathbb{P}(X \in \{1, 3, 5\}) = 3/6$$

- B is the event of getting a number smaller than 4

$$\mathbb{P}(X \in \{1, 2, 3\}) = 3/6$$

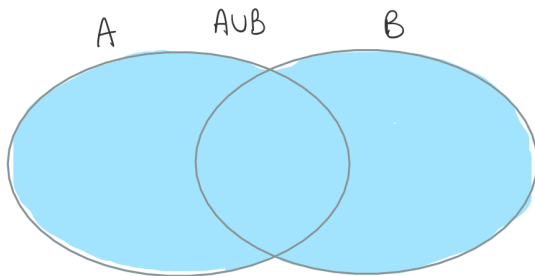
Then $\mathbb{P}(A \cap B) = \mathbb{P}(X \in \{1, 3\}) = 2/6$

Probability of A or B

$$\mathbb{P}(A \text{ or } B) = \mathbb{P}(A \cup B) = \mathbb{P}(A) + \mathbb{P}(B) - \mathbb{P}(A \cap B)$$

where:

- $\mathbb{P}(A \cup B)$ is the probability that at least one of A or B occurs.
- The subtraction of $\mathbb{P}(A \cap B)$ avoids double-counting the overlap.



$$\mathbb{P}(A \text{ or } B) = \mathbb{P}(A \cup B) = \mathbb{P}(A) + \mathbb{P}(B) - \mathbb{P}(A \cap B)$$

Example: You're throwing a die and

- A is the event of getting an even number

$$\mathbb{P}(X \in \{2, 4, 6\}) = 3/6$$

- B is the event of getting a number bigger than 3

$$\mathbb{P}(X \in \{4, 5, 6\}) = 3/6$$

Then $\mathbb{P}(A \cup B) = \mathbb{P}(X \in \{2, 4, 5, 6\}) = 3/6 + 3/6 - 2/6 = 4/6$



Independent events

Two events A and B are said to be **independent** if and only if

$$\mathbb{P}(A \cap B) = \mathbb{P}(A) \mathbb{P}(B)$$

where:

- $\mathbb{P}(A)$ and $\mathbb{P}(B)$ are the probabilities of events A and B , respectively.
- Independence means that the occurrence of one event does not affect the probability of the other.



Example: You're throwing two dice and

- A is the event of getting an even number on the first one

$$\mathbb{P}(X_1 \in \{2, 4, 6\}) = 3/6$$

- B is the event of getting an odd number on the second one

$$\mathbb{P}(X_2 \in \{1, 3, 5\}) = 3/6$$

Then $\mathbb{P}(A \cap B) = \mathbb{P}(X_1 \in \{2, 4, 6\}, X_2 \in \{1, 3, 5\}) = 3/6 * 3/6 = 9/36$



	1	2	3	4	5	6
1	(1,1)	(1,2)	(1,3)	(1,4)	(1,5)	(1,6)
2	(2,1)	(2,2)	(2,3)	(2,4)	(2,5)	(2,6)
3	(3,1)	(3,2)	(3,3)	(3,4)	(3,5)	(3,6)
4	(4,1)	(4,2)	(4,3)	(4,4)	(4,5)	(4,6)
5	(5,1)	(5,2)	(5,3)	(5,4)	(5,5)	(5,6)
6	(6,1)	(6,2)	(6,3)	(6,4)	(6,5)	(6,6)

Conditional Probability

$$\mathbb{P}(A \mid B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)}$$

where:

- $\mathbb{P}(B) > 0$
- A and B are events in the same probability space

Idea: we are looking at the probability of A occurring given that we know that B has already happened



Example: roll a die and

- A is the event of getting an even number

$$\mathbb{P}(A) = \mathbb{P}(X \in \{2, 4, 6\}) = 3/6$$

- B is the event of getting a number less than 4

$$\mathbb{P}(B) = \mathbb{P}(X \in \{1, 2, 3\}) = 3/6$$

The intersection $A \cap B$ corresponds to getting an even number less than 4:

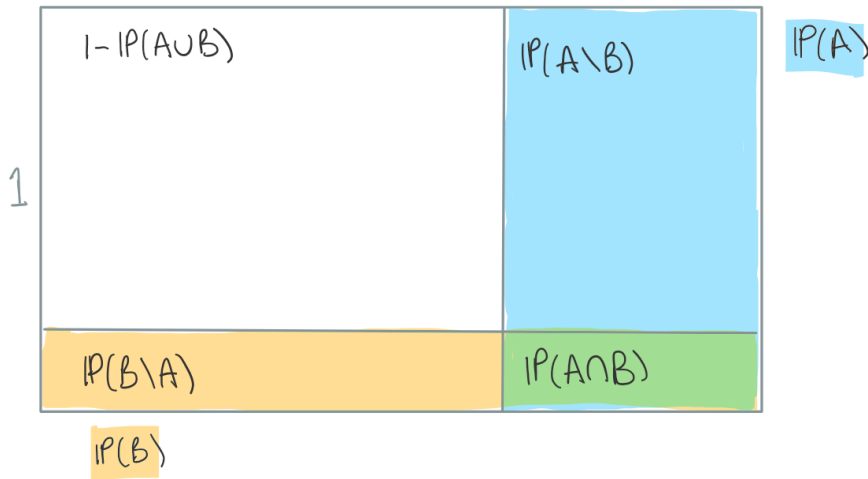
$$A \cap B = \{2\} \quad \Rightarrow \quad \mathbb{P}(A \cap B) = 1/6.$$

Therefore,

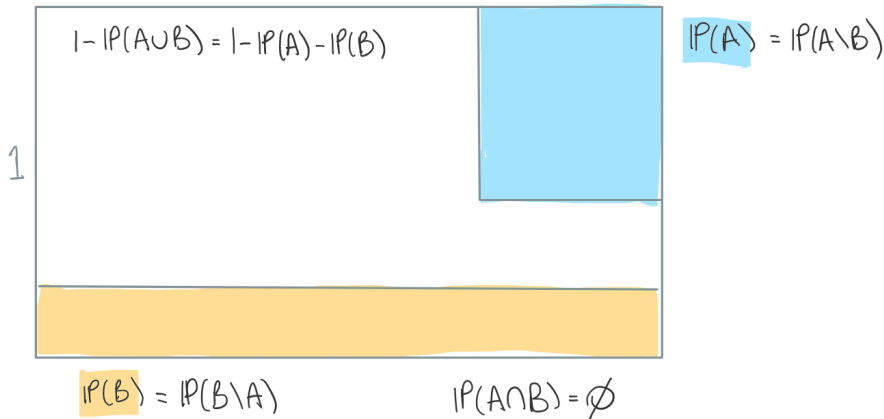
$$\mathbb{P}(A \mid B) = \frac{1/6}{3/6} = \frac{1}{3}.$$



$$\mathbb{P}(A | B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)}$$



$$\mathbb{P}(A | B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)} = 0$$



Something interesting we can notice is that

$$\mathbb{P}(A \mid B) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(B)} \quad \text{and} \quad \mathbb{P}(B \mid A) = \frac{\mathbb{P}(A \cap B)}{\mathbb{P}(A)}.$$

and rearranging these terms gives

$$\mathbb{P}(A \cap B) = \mathbb{P}(A \mid B) \mathbb{P}(B) = \mathbb{P}(B \mid A) \mathbb{P}(A).$$

We can use this to derive a very important theorem

Bayes' Theorem

$$\mathbb{P}(A | B) = \frac{\mathbb{P}(B | A) \mathbb{P}(A)}{\mathbb{P}(B)}$$

where

- $\mathbb{P}(B) > 0$
- A and B are events in the same probability space.

Idea: the theorem shows the relationship between the two conditional probabilities

Why do we care about all this?

We can use this formula to create the most basic of classifiers for any number of categories

Naive Bayes

Naive Bayes is a probabilistic classifier based on **Bayes' theorem**

For a class C and feature vector $X = (x_1, x_2, \dots, x_n)$:

$$P(C | X) = \frac{P(C) \cdot P(X | C)}{P(X)}$$

in our case $C \in \{\text{setosa, versicolor, virginica}\}$ and X are features sepal length, sepal width, petal length, petal width

Idea: we want to find each of the probabilities $P(C | X)$ for every category C , because they represent the probability that a certain observation is of category C given the data X

Example: if petal length is 5.2cm and sepal width is 2.9cm, the probability that this flower is setosa might be 0.6, while the probability that it is versicolor might be 0.3 and virginica 0.1

$$P(C | X) = \frac{P(C) \cdot P(X | C)}{P(X)}$$

where:

- $P(C | X)$: probability of the obs being of class C given we know its features X
- $P(C)$: general probability of class C occurring
- $P(X | C)$: distribution of features X given class C
- $P(X)$: probability of features X

Idea: we need to deal with these three elements to find out target probabilities!



Idea: we can ignore $P(X)$ at the denominator because it appears in all probabilities, and in the end we can just reweigh them to make sure the add up to 1!

$$P(C | X) \propto P(C)P(X | C)$$

Idea: we can calculate $P(C)$ directly from the dataset, since we assume it is a good representation of reality (it's called naive for a reason...)

$$P(C) = \frac{\text{Number of samples in class } C}{\text{Total number of samples}} = \frac{N_C}{N}$$

For our dataset we have a simple case because they are all the same

$$P(\text{setosa}) = \frac{50}{150} = 0.3333$$

$$P(\text{versicolor}) = \frac{50}{150} = 0.3333$$

$$P(\text{virginica}) = \frac{50}{150} = 0.3333$$



Finally we move to $P(X \mid C)$

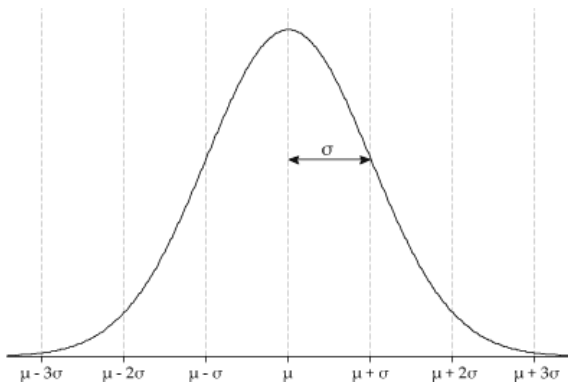
Idea: we can assume that all features are independent (again, naive, but it works!), so the probability will just be the product of the others

$$P(X \mid C) = \prod_{i=1}^n P(x_i \mid C)$$

Naive Bayes

For **continuous features** like in this case, we make a further assumption that they have a **gaussian distribution**

$$P(x_i | C) = \frac{1}{\sqrt{2\pi\sigma_C^2}} \exp\left(-\frac{(x_i - \mu_C)^2}{2\sigma_C^2}\right)$$



We **estimate** mean and variance from the dataset like we did for $P(C)$

$$\text{Mean: } \mu_{C,i} = \frac{1}{N_C} \sum_{j=1}^{N_C} x_{j,i}$$

$$\text{Variance: } \sigma_{C,i}^2 = \frac{1}{N_C - 1} \sum_{j=1}^{N_C} (x_{j,i} - \mu_{C,i})^2$$

Idea: you only use the values of the observations on the dataset that belong to the specific category C

Example: Compute the probability that sepal length $x_i = 5.2$ given the species C is Setosa

$$P(x_i = 5.2 \mid C = \text{Setosa}) = \frac{1}{\sqrt{2\pi\sigma_{\text{Setosa}}^2}} \exp\left(-\frac{(5.2 - \mu_{\text{Setosa}})^2}{2\sigma_{\text{Setosa}}^2}\right)$$

From dataset: $\mu_{Setosa} = 5.01$, $\sigma_{Setosa}^2 = 0.12$

$$\begin{aligned} P(x_i = 5.2 \mid C = \text{Setosa}) &\approx \frac{1}{\sqrt{2\pi \cdot 0.12}} \exp\left(-\frac{(5.2 - 5.01)^2}{2 \cdot 0.12}\right) \\ &\approx \frac{1}{0.868} \cdot e^{0.150} \approx 0.991 \end{aligned}$$

Remember: a probability density can have any value even higher than 1, what's important is that the integral of it all is 1

For **discrete features** with multiple categories, we use a **multinomial distribution**.

The probability of each feature becomes

$$P(\underline{x}_i \mid C) = \frac{n!}{x_{i1}! x_{i2}! \cdots x_{im}!} p_{i1}^{x_{i1}} p_{i2}^{x_{i2}} \cdots p_{im}^{x_{im}}$$

Where each feature has m categories $\underline{x}_i = (x_{i1}, x_{i2}, \dots, x_{im})$

Idea: same as before, this is the distribution of the count of the faces of a dice roll

Naive Bayes

Here is what the code looks like

```
from sklearn.naive_bayes import GaussianNB
#train
gnb = GaussianNB()
gnb.fit(X_train, y_train)

#predict
y_pred = gnb.predict(X_test)
```

Very easy to implement!

In our Iris example, we get an accuracy of 0.977

Very high! But I picked this dataset for the example for a reason

Note: these models are good also to get a benchmark model to compare other models too, because how easy they are to set up!

Thank you for following! We will now move to other models